

SESIÓN 2 · CLAUDE CERTIFIED ARCHITECT

Varios agentes, un solo sistema.

Multi-agent, Agentic RAG y gestión de sesiones: coordinar agentes, buscar con criterio y no perder el hilo de la conversación.

Lo que quedó de la sesión 1

Agente

Modelo + herramientas + loop.
Decide qué paso sigue.

Workflow vs agente

Si el flujo es fijo y conocido, no
necesitás un agente.

Human-in-the-loop

Checkpoints humanos donde el
error duele.

Hoy: qué pasa cuando **un solo agente no alcanza**.

Hoy

01

Multi-agent

Patrones para coordinar varios agentes sin que se pisen

02

Agentic RAG

El retrieval deja de ser una búsqueda fija

03

Sesiones y estado

El modelo es stateless: quién sostiene la conversación

**Un agente puede hacer todo.
Eso no significa que deba
hacerlo.**

¿Por qué multi-agente?

Contexto

Un agente que hace todo arrastra todo. La ventana se llena de cosas que no usa.

Especialización

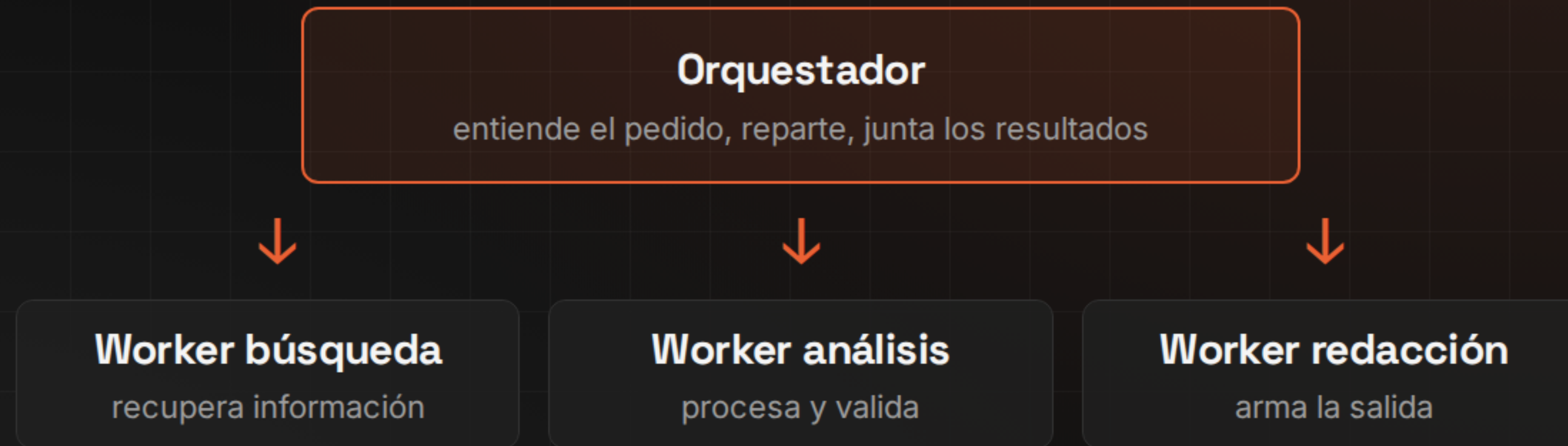
Un rol claro con pocas herramientas rinde más que un prompt gigante.

Paralelismo

Tareas independientes pueden correr a la vez.

La contra: más costo, más latencia, más cosas que se rompen. **Multi-agente se justifica, no se asume.**

Patrón 1: orquestador y workers



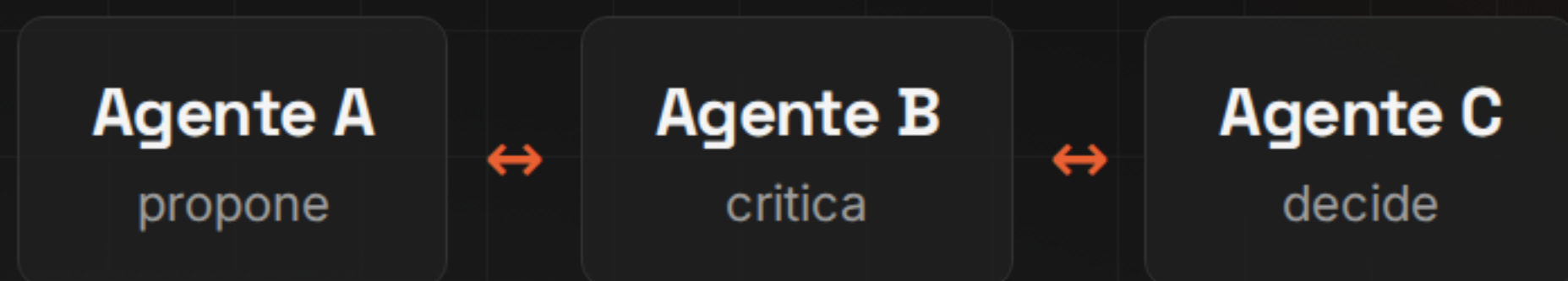
Cada worker hace una sola cosa bien. Los resultados **vuelven al orquestador**, que arma la respuesta final.

Patrón 2: pipeline



La salida de uno es la entrada del siguiente. Orden fijo, **fácil de debuggear**: sabés en qué paso se rompió.

Patrón 3: peer-to-peer



Sin jerarquía fija: cualquiera le habla a cualquiera. El más flexible y **el más difícil de controlar**.

Cuándo usar cada uno

PATRÓN	CUÁNDO	RIESGO
Orquestador y workers	Tareas variadas que hay que repartir	El orquestador es cuello de botella
Pipeline	Proceso conocido, pasos fijos	Rígido: un paso malo arrastra al resto
Peer-to-peer	Problemas abiertos, agentes que negocian	Difícil de debuggear y de acotar

Si dudás, **arrancá con orquestador y workers.**

El contrato de cada agente

DEFINICIÓN DE UN WORKER

Rol: triage de tickets de soporte

Recibe: ticket crudo + historial del cliente

Devuelve: { categoria, prioridad, resumen } en JSON

Puede: buscar en la wiki, leer el CRM

No puede: responder al cliente, escribir en la DB

Si no podés escribir esto en cinco líneas, **el agente está mal recortado**.

Qué se pasan los agentes

SE PASA

Resultados estructurados: JSON, no prosa
Referencias a datos: un id, no el documento entero
Lo mínimo que el paso siguiente necesita

NO SE PASA

El historial completo de la conversación
Contexto crudo "por las dudas"
Tool results enteros sin resumir

Pasá **resultados**, no conversaciones.

Cuando un agente falla

Falla técnica

Timeout o error de tool. Retry con límite y fallback definido.

Alucinación

Devuelve algo inventado. El output se valida contra schema antes de usarse.

Silencio

No responde. El orquestador tiene timeout propio: no espera para siempre.

El plan para cada falla se diseña **antes**. En producción es tarde.

El precio de dividir

Tokens

Cada agente paga su system prompt y su contexto. Dividir multiplica.

Latencia

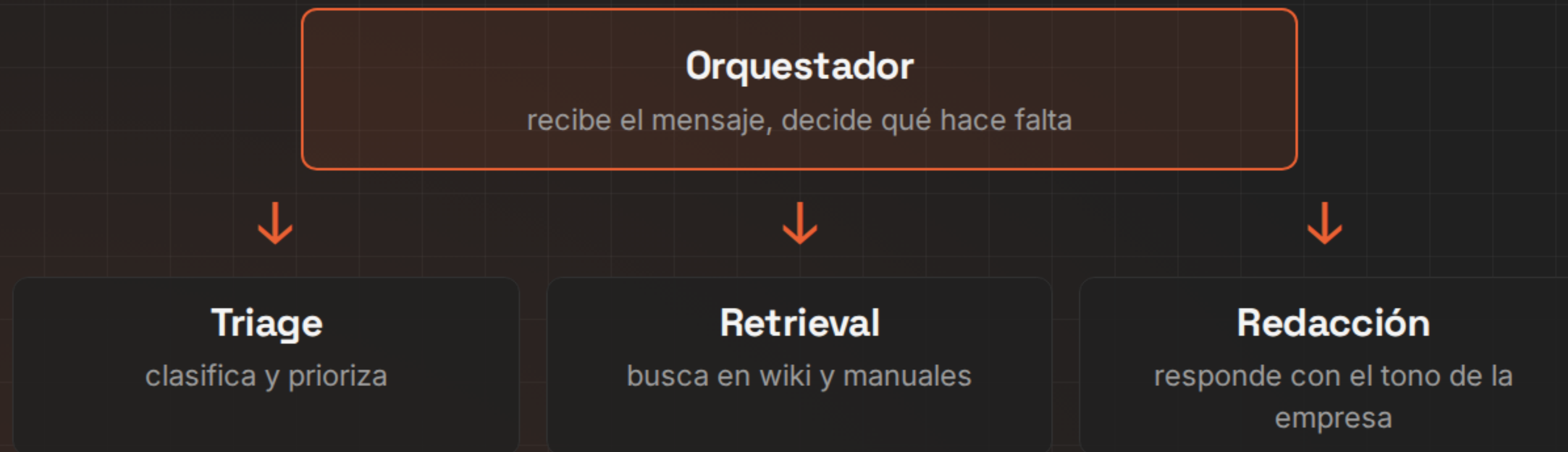
Los pasos se encadenan: la respuesta tarda la suma.

Debug

Un error puede nacer en un agente y explotar en otro.

Medí el sistema con un agente **antes de partirlo en cinco**.

Caso: asistente de soporte



Arquitectura real, simplificada. Cada worker con su contrato de cinco líneas y **sin acceso a lo que no necesita**.

RAG clásico, repaso en un diagrama



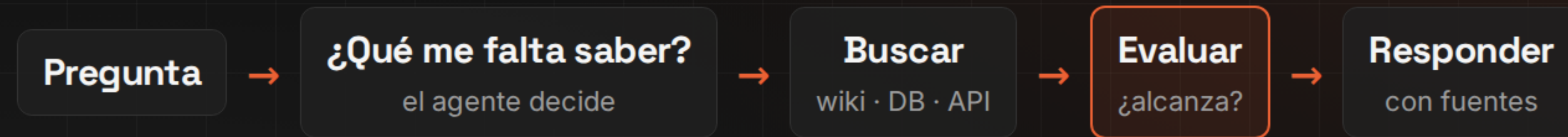
Una búsqueda, definida por código. **El modelo no decide nada.**

**El RAG clásico busca una vez.
Si buscó mal, responde mal.**

¿Qué es Agentic RAG?

El agente decide qué, cuándo y dónde buscar

El flujo agéntico



↪ no alcanza: reformular la query o cambiar de fuente

El retrieval deja de ser un paso fijo y pasa a ser **una herramienta más del agente**.

Qué cambia respecto al clásico

Retrieval iterativo

Busca, mira qué vino, vuelve a buscar con mejor query.

Multi-fuente

Wiki, base de datos, APIs externas. Elige dónde según la pregunta.

Re-ranking

De 20 chunks recuperados pasan 3: los que responden la pregunta.

Validación

Responde citando fuente. Sin fuente, dice que no sabe.

Caso: wiki de soporte técnico

LO QUE HACE EL AGENTE POR ADENTRO

Usuario: el lector de tarjetas no lee, ¿qué hago?

1. buscar("lector no lee tarjeta") → 12 resultados, ninguno del modelo X
2. buscar("lector X troubleshooting") → doc oficial del fabricante
3. evaluar → cubre el caso, versión vigente
4. responder con los pasos + link a la fuente

Dos búsquedas en vez de una. La diferencia entre "no encontré nada" y **una respuesta útil**.

Errores comunes en RAG agéntico

✗ Over-retrieval: traer 40 chunks por las dudas y saturar el contexto

✗ Sin fuente de verdad: dos docs se contradicen y el agente elige al azar

✗ Loop infinito: re-buscar sin límite de intentos

✗ Indexar todo: PII y secretos adentro del vector store

Antes de implementar

☐ ¿Qué fuentes entran y quién las mantiene?

☐ ¿Qué responde si no encuentra nada?

☐ ¿Cómo medís que responde bien?

☐ ¿Cada cuánto se re-indexa?

☐ ¿Cuántas búsquedas máximo por pregunta?

☐ ¿Qué NO debería estar indexado?

**El modelo es stateless.
La conversación la sostenés
vos.**

Contexto, sesión, memoria

CONTEXT0

Lo que el modelo ve en esta llamada.

Vive en la ventana, muere con ella.

SESIÓN

Una conversación con principio y fin.

Vive en tu backend mientras dura.

MEMORIA

Lo que sobrevive entre sesiones.

Vive en una DB, se trae cuando sirve.

Tres cosas distintas. Si las mezclás, **el bug aparece solo.**

Dónde vive cada cosa

QUÉ	DÓNDE	POR QUÉ
Turnos recientes	cache (Redis)	rápido, se descarta al cerrar
Conversación completa	base de datos	auditoría y análisis
Conocimiento del usuario	DB + vector store	memoria de largo plazo
Documentos y wiki	vector store	retrieval semántico

Cuando la conversación crece

Truncar

Cortás los turnos viejos. Barato.
Perdés lo que dijeron al principio.

Resumen progresivo

Comprimís lo viejo en un resumen
que se va actualizando. Cuesta
una llamada extra, no pierde
decisiones.

Extracción selectiva

Guardás solo datos duros y
decisiones. Lo demás se va.

El historial completo no entra. **Elegí qué perder.**

Caso: bot de WhatsApp, muchas conversaciones

Usuario A → sesión A • historial + estado + memoria

Usuario B → sesión B • historial + estado + memoria

Usuario C → sesión C • historial + estado + memoria

LA CLAVE

session_id = número de teléfono. Todo lo que persiste cuelga de ahí.

EL BUG CLÁSICO

Contexto compartido entre sesiones: el usuario A ve datos del usuario B.

Retomar a un usuario

LO QUE SE CARGA AL ARRANCAR LA SESIÓN

usuario: 8841

memoria → prefiere factura A, ya reportó el bug del lector

resumen → conversaciones anteriores, comprimidas

historial → solo los últimos turnos de esta sesión

Continuidad no es guardar todo. Es **traer poco y bien elegido**.

Errores al combinar todo

- × Multi-agente porque suena bien: un agente alcanzaba

- × Sesión infinita sin resumen: la ventana explota a los 50 turnos

- × Cada worker con su propio RAG: contexto duplicado, factura doble

- × Estado importante solo en la ventana: se corta la sesión y no queda nada

Para llevarse

Multi-agente solo cuando un agente no alcanza

Patrones orquestador, pipeline, peer-to-peer: cada uno con su caso

Agentic RAG el agente decide qué buscar y evalúa lo que encuentra

Sesiones el estado vive afuera del modelo, con dueño claro

**Más agentes no es
mejor arquitectura.
Mejor coordinación, sí.**

Próxima sesión: diseño de ruta técnica MCP.

Todas estas piezas necesitan herramientas. MCP es cómo se conectan sin escribir un conector por cada una.